

Project Robust Statistics

2026-05-03

We import libraries:

```
library(moments)
library(corrplot)
library(car)
library(robustHD)
library(robustbase)
library(mrfDepth)
library(readxl)
library(psych)
library(corrplot)
library(plot3D)
library(MASS)
library(Hmisc)
library(moments)
library(rrcov)
library(factoextra)
library(dplyr)
library(cluster)
library(corrplot)
library(fossil)
library(stats)
library(cellwise)
```

We now introduce the dataset:

```
load("cancer_data.Rdata")
set.seed(1085999)
rsub=sample(3047)[1:500]
data=cancer_data[rsub, ]
```

```
load("data_USCancer.Rdata")
originalfull=data_USCancer
datanames=originalfull[rsub,]
data$countyname=datanames$Geography
```

We start with an exploratory data analysis to have a better insight of our dataset. We start with a summary:

```
summary(data)
```

```
##      avganncount      avgdeathsperyear target_deathrate incidencerate
## Min.      :    6.0   Min.      :    3.0   Min.      :105.4   Min.      :234.0
```

```
## 1st Qu.: 70.0 1st Qu.: 27.0 1st Qu.:162.9 1st Qu.:420.8
## Median : 155.0 Median : 52.0 Median :179.7 Median :453.5
## Mean : 512.8 Mean : 146.3 Mean :179.9 Mean :449.6
## 3rd Qu.: 467.0 3rd Qu.: 124.2 3rd Qu.:195.9 3rd Qu.:480.9
## Max. :11217.0 Max. :3927.0 Max. :277.6 Max. :587.4
## medincome popest2015 povertyp percent medianage
## Min. : 24.04 Min. : 1.13 Min. : 4.00 Min. : 26.40
## 1st Qu.: 37.53 1st Qu.: 10.87 1st Qu.:12.47 1st Qu.: 37.80
## Median : 44.38 Median : 22.37 Median :16.30 Median : 40.90
## Mean : 46.00 Mean : 77.19 Mean :17.37 Mean : 44.99
## 3rd Qu.: 51.93 3rd Qu.: 62.67 3rd Qu.:21.52 3rd Qu.: 43.80
## Max. :108.48 Max. :2636.74 Max. :40.60 Max. :535.20
## medianagemale medianagefemale percentmarried pctnohs18_24
## Min. :26.10 Min. :26.60 Min. :27.60 Min. : 0.50
## 1st Qu.:36.38 1st Qu.:39.17 1st Qu.:47.30 1st Qu.:13.30
## Median :39.50 Median :42.40 Median :52.20 Median :17.85
## Mean :39.69 Mean :42.28 Mean :51.44 Mean :18.70
## 3rd Qu.:42.60 3rd Qu.:45.33 3rd Qu.:56.40 3rd Qu.:23.00
## Max. :58.60 Max. :56.80 Max. :67.00 Max. :59.70
## pcths18_24 pctbachdeg18_24 pcths25_over pctbachdeg25_over
## Min. : 8.60 Min. : 0.000 Min. : 8.30 Min. : 2.50
## 1st Qu.:29.38 1st Qu.: 2.700 1st Qu.:31.07 1st Qu.: 9.00
## Median :35.00 Median : 5.300 Median :36.20 Median :11.50
## Mean :35.21 Mean : 6.024 Mean :35.45 Mean :12.65
## 3rd Qu.:40.73 3rd Qu.: 8.025 3rd Qu.:39.80 3rd Qu.:15.22
## Max. :72.50 Max. :51.800 Max. :52.10 Max. :34.50
## pctunemployed16_over pctprivatecoverage pctempprivcoverage pctpubliccoverage
## Min. : 0.800 Min. :29.60 Min. :17.70 Min. :14.00
## 1st Qu.: 5.500 1st Qu.:56.50 1st Qu.:34.08 1st Qu.:31.90
## Median : 8.000 Median :64.45 Median :40.80 Median :37.30
## Mean : 8.055 Mean :63.69 Mean :40.63 Mean :37.00
## 3rd Qu.:10.000 3rd Qu.:71.72 3rd Qu.:46.83 3rd Qu.:42.33
## Max. :24.600 Max. :89.60 Max. :66.70 Max. :58.10
## pctpubliccoveragealone pctwhite pctblack pctasian
## Min. : 4.20 Min. : 17.45 Min. : 0.0000 Min. : 0.0000
## 1st Qu.:15.30 1st Qu.: 75.42 1st Qu.: 0.6159 1st Qu.: 0.2323
## Median :19.65 Median : 90.04 Median : 2.2169 Median : 0.4979
## Mean :19.76 Mean : 82.61 Mean : 9.8860 Mean : 1.0837
## 3rd Qu.:23.93 3rd Qu.: 95.33 3rd Qu.:11.0063 3rd Qu.: 1.0066
## Max. :40.60 Max. :100.00 Max. :72.0249 Max. :33.8295
## pctotherrace pctmarriedhouseholds birthrate
## Min. : 0.0000 Min. :24.43 Min. : 0.000
## 1st Qu.: 0.2875 1st Qu.:47.01 1st Qu.: 4.610
## Median : 0.8355 Median :51.54 Median : 5.413
## Mean : 1.9470 Mean :50.73 Mean : 5.785
## 3rd Qu.: 2.0887 3rd Qu.:55.10 3rd Qu.: 6.632
## Max. :41.9303 Max. :70.27 Max. :17.877
```

We can see that the variables are on different scales.

Then we check the skewness of our data:

```
for (i in 1:27)
{
```

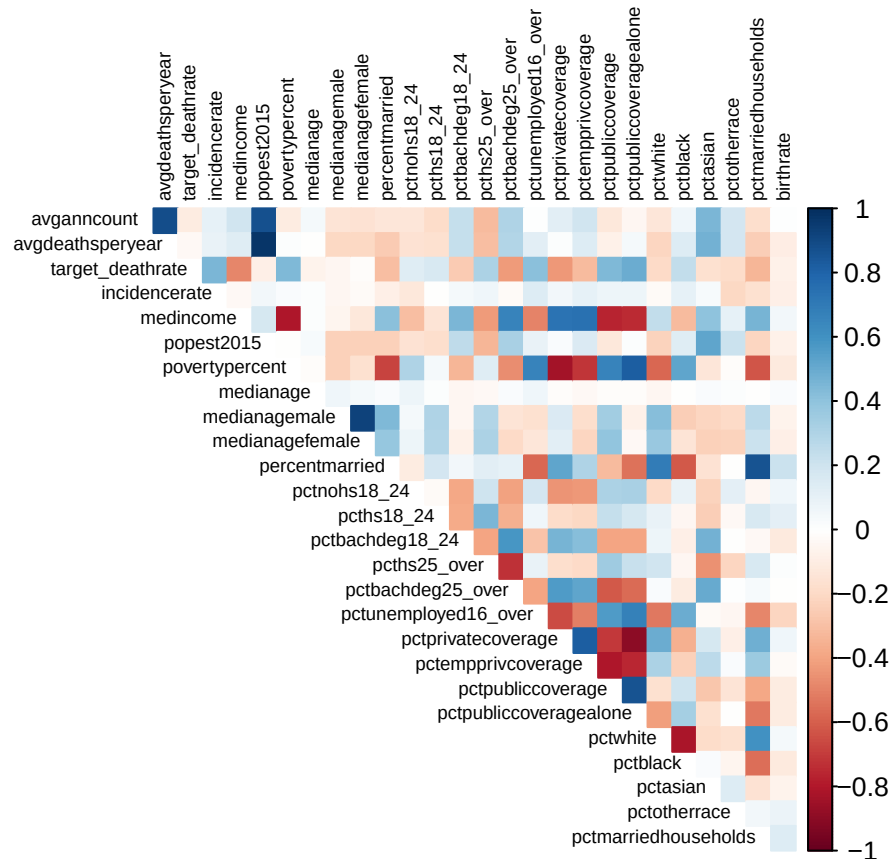
```
print(skewness(data[,i]))
}
```

```
## [1] 5.749139
## [1] 7.254746
## [1] 0.2382024
## [1] -0.5650305
## [1] 1.482265
## [1] 7.793565
## [1] 0.6892345
## [1] 10.03322
## [1] 0.3326684
## [1] -0.1543359
## [1] -0.5546824
## [1] 1.138839
## [1] 0.3830275
## [1] 2.705737
## [1] -0.5172791
## [1] 1.113734
## [1] 0.7179889
## [1] -0.376693
## [1] 0.05371693
## [1] -0.03965421
## [1] 0.3874094
## [1] -1.512219
## [1] 1.977581
## [1] 8.12181
## [1] 5.751253
## [1] -0.4373016
## [1] 1.229242
```

We can see that many variables have a strong right-skewness, on the other hand only one has strong left-skewness. We should apply some transformations to decrease the skewness but before let's continue with the exploratory data analysis.

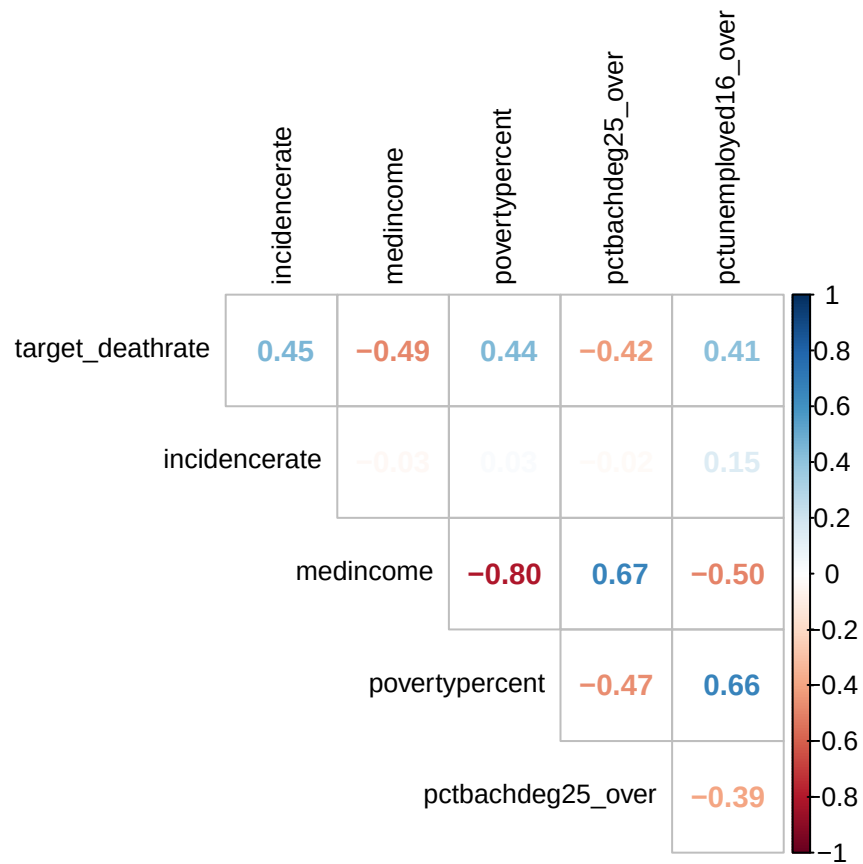
We now plot corplot to see if the variables are correlated (it is probably better to not consider all the variables but study them by group because if not the graph would become too difficult to study):

```
numdata=data[,sapply(data, is.numeric)]
correlation=cor(numdata, use='pairwise.complete.obs')
corrplot(correlation, method='color', type='upper', tl.col='black', tl.cex=0.6, diag=FALSE)
```

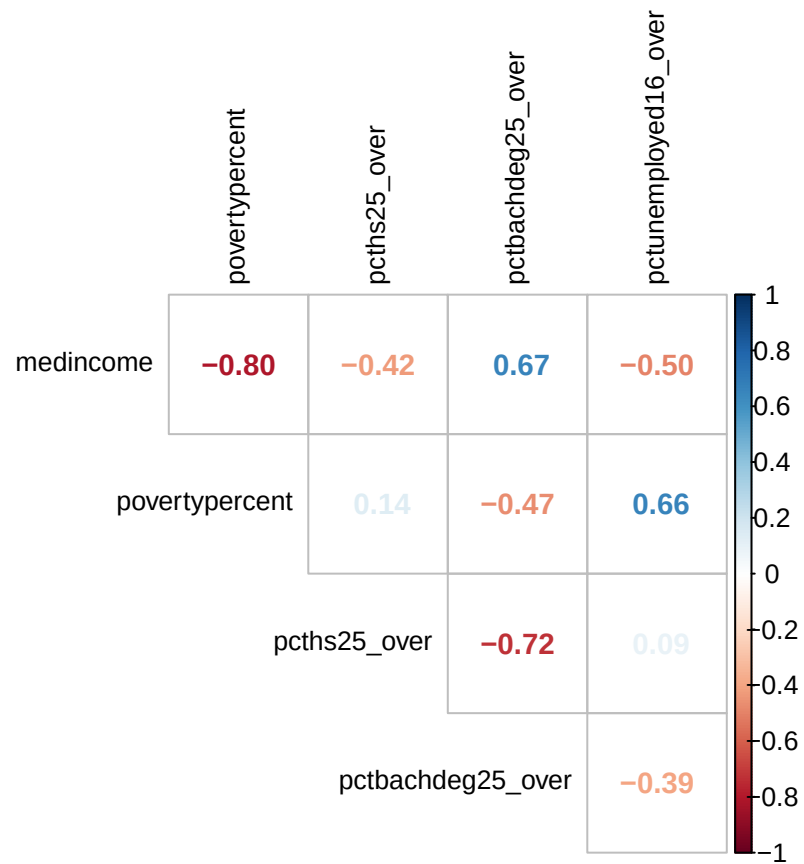


From this we are able to see which one are almost not correlated and which are strongly correlated, so now we can consider them in different groups to make our study easier.

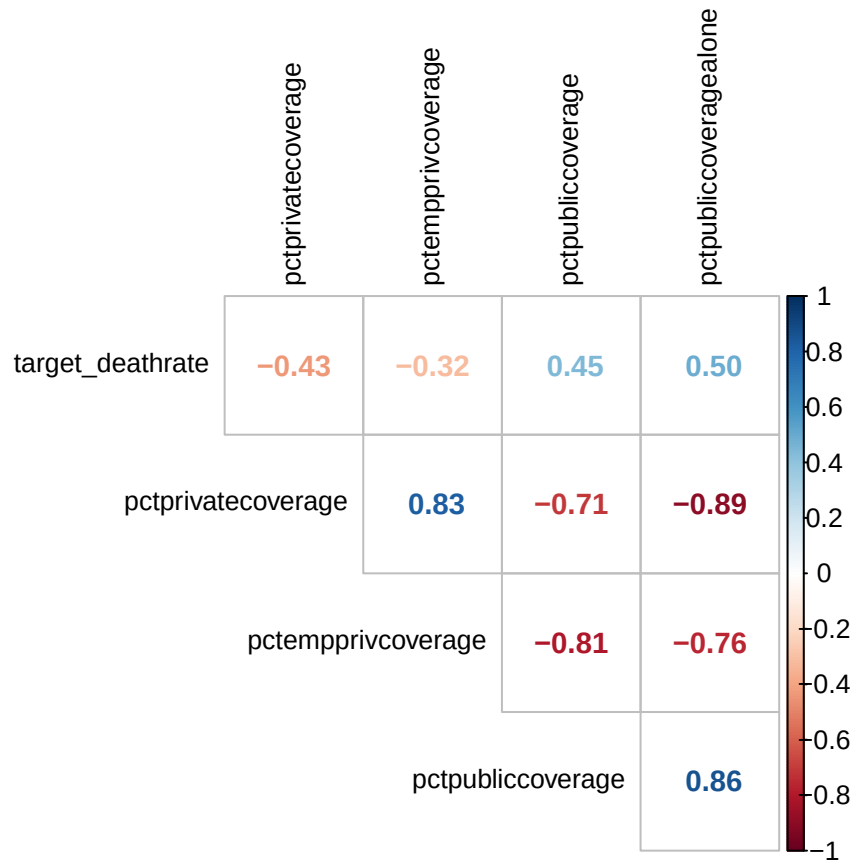
```
varcore=c("target_deathrate", "incidencerate", "medincome", "povertypercent", "pctbachdeg25_over", "pct  
correlation1=cor(data[, varcore], use='pairwise.complete.obs')  
corrplot(correlation1, method='number', type='upper', tl.col='black', tl.cex=0.8, diag=FALSE, number.cex=
```



```
varsocio=c("medincome", "povertypercent", "pcths25_over", "pctbachdeg25_over", "pctunemployed16_over")
correlation2=cor(data[, varsocio], use='pairwise.complete.obs')
corrplot(correlation2, method='number', type='upper', tl.col='black', tl.cex=0.8, diag=FALSE, number.cex=0.8)
```

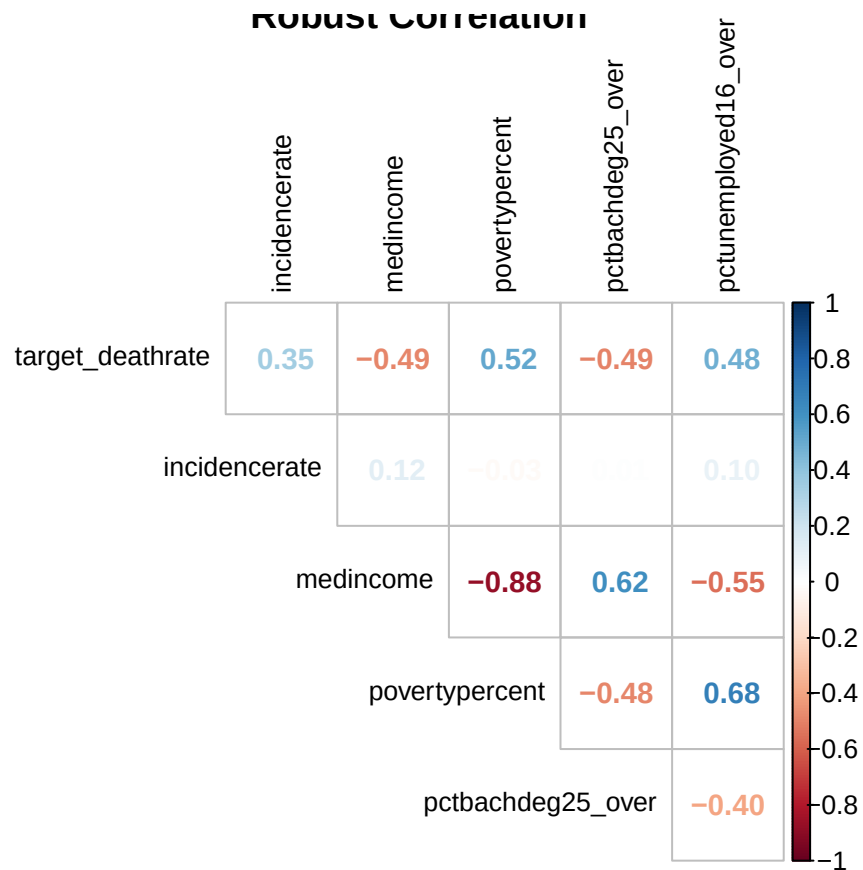


```
varhealth=c("target_deathrate", "pctprivatecoverage", "pctempprivcoverage", "pctpubliccoverage", "pctunemployed16_over")
correlation3=cor(data[, varhealth], use='pairwise.complete.obs')
corrplot(correlation3, method='number', type='upper', tl.col='black', tl.cex=0.8, diag=FALSE, number.cex=0.8)
```

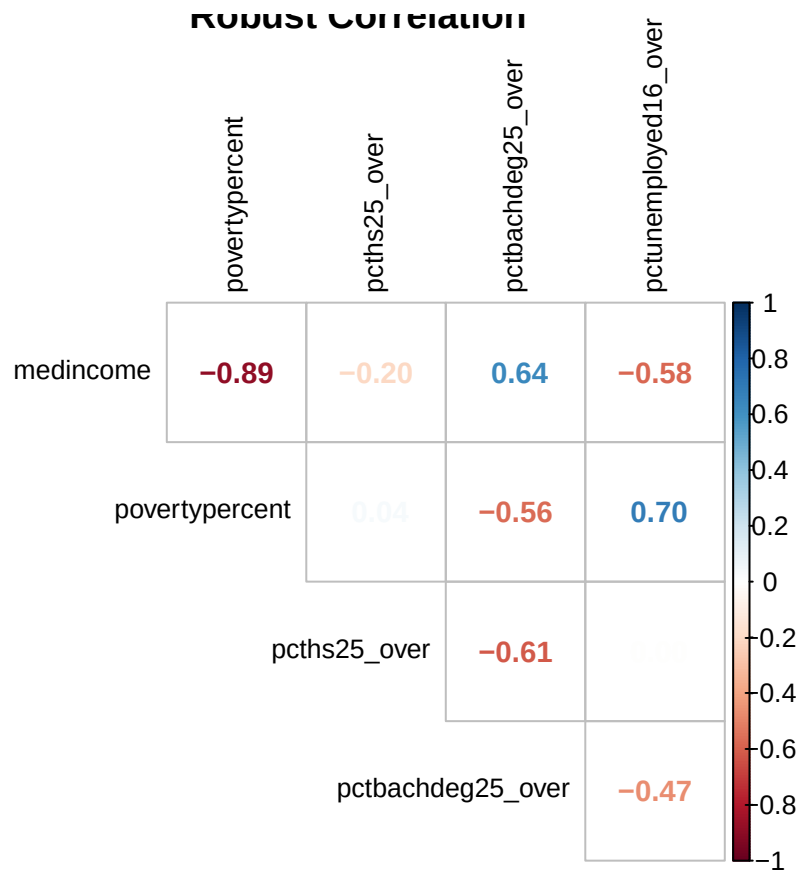


We know that the classic correlation is extremely sensible to outliers so we will try also with the robust one:

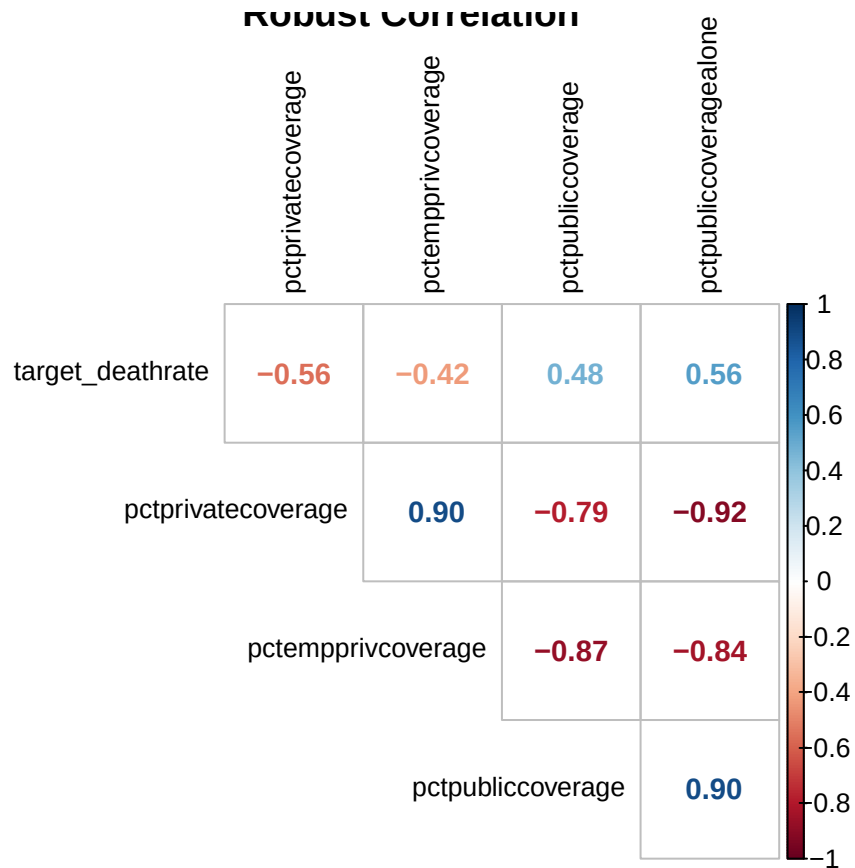
```
robustcov=covMcd(data[, varcore])
robustcor=cov2cor(robustcov$cov)
corrplot(robustcor, method='number', type='upper', tl.col='black', tl.cex=0.8, diag=FALSE, number.cex=0.8)
```



```
robustcovsocio=covMcd(data[, varsocio])
robustcorsocio=cov2cor(robustcovsocio$cov)
corrplot(robustcorsocio, method='number', type='upper', tl.col='black', tl.cex=0.8, diag=FALSE, number.
```

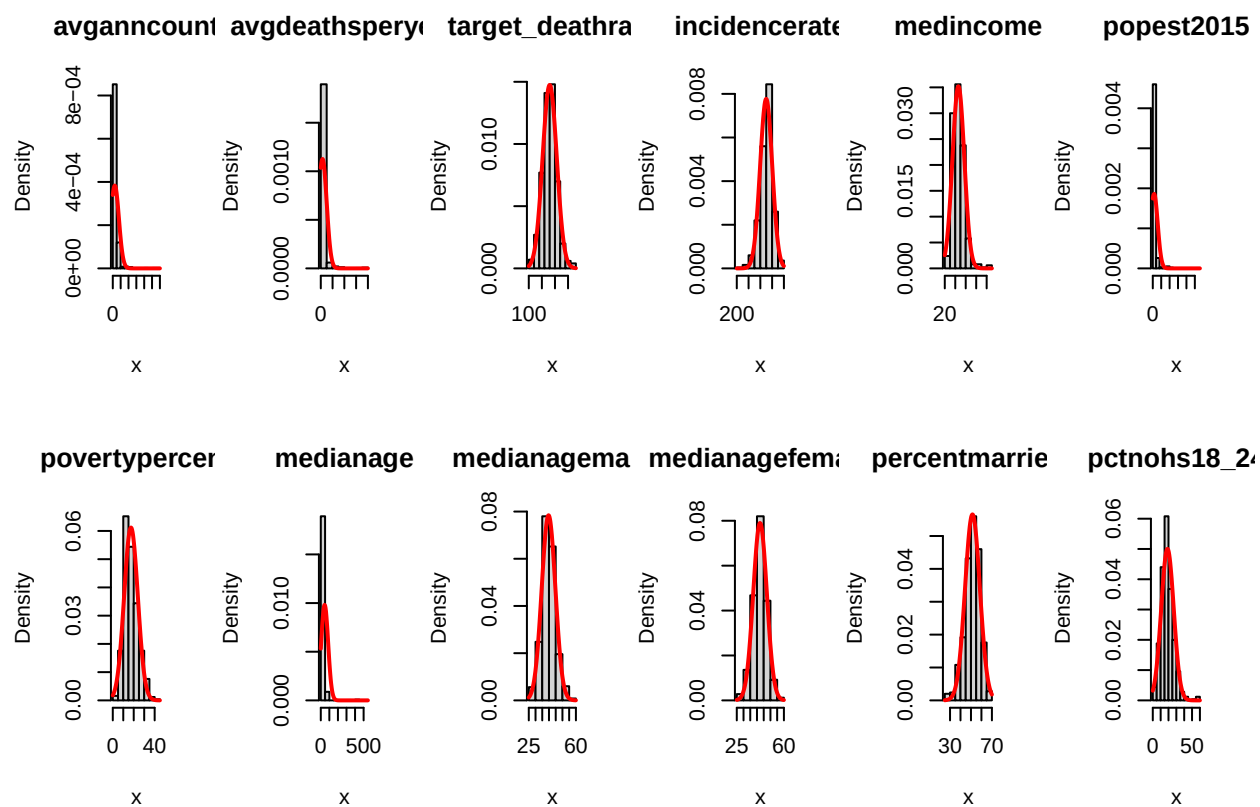


```
robustcovhealth=covMcd(data[, varhealth])
robustcorhealth=cov2cor(robustcovhealth$cov)
corrplot(robustcorhealth, method='number', type='upper', tl.col='black', tl.cex=0.8, diag=FALSE, number
```

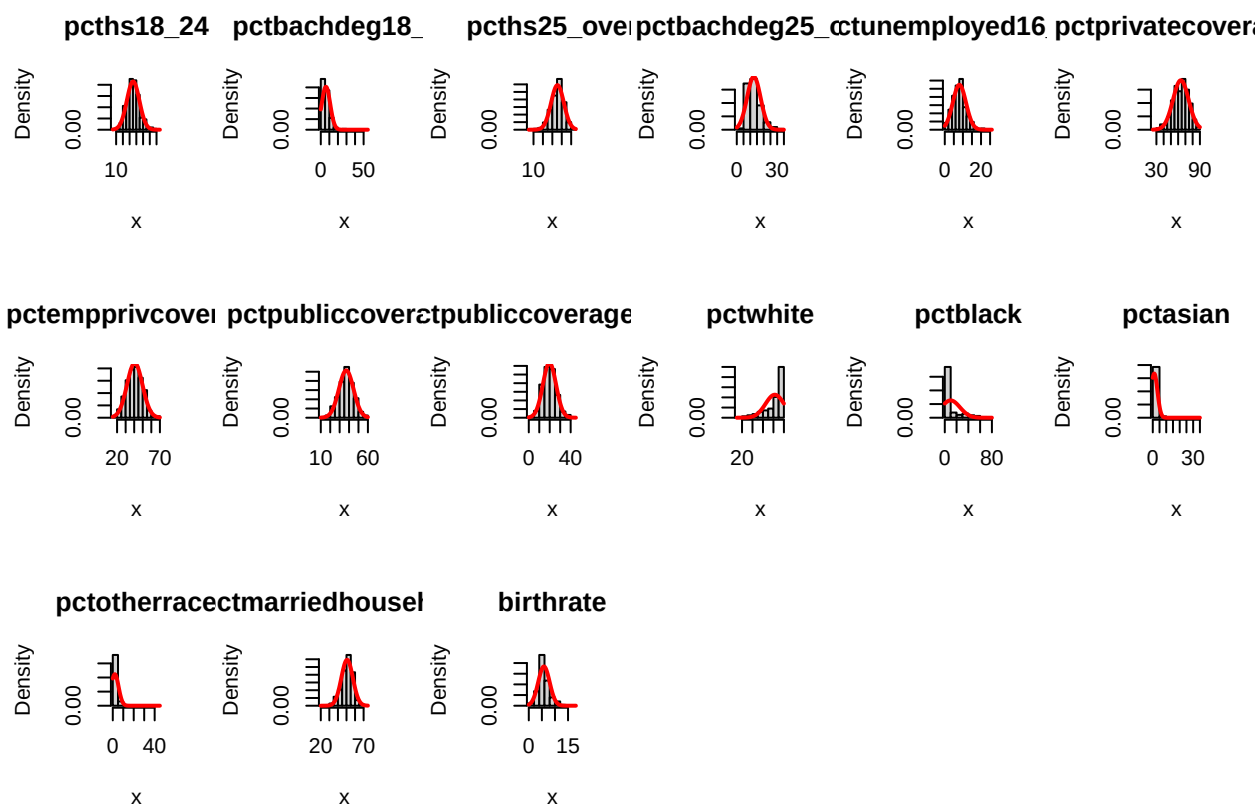


We now plot histograms and then QQ-plots to check normality:

```
par(mfrow=c(2,6))
for (i in 1:12)
{
  x=data[,i]
  hist(x, freq=FALSE, main=colnames(data)[i])
  mn=mean(x, na.rm=TRUE)
  std=sd(x, na.rm=TRUE)
  curve(dnorm(x, mean=mn, sd=std), col='red', lwd=2, add=TRUE)
}
```

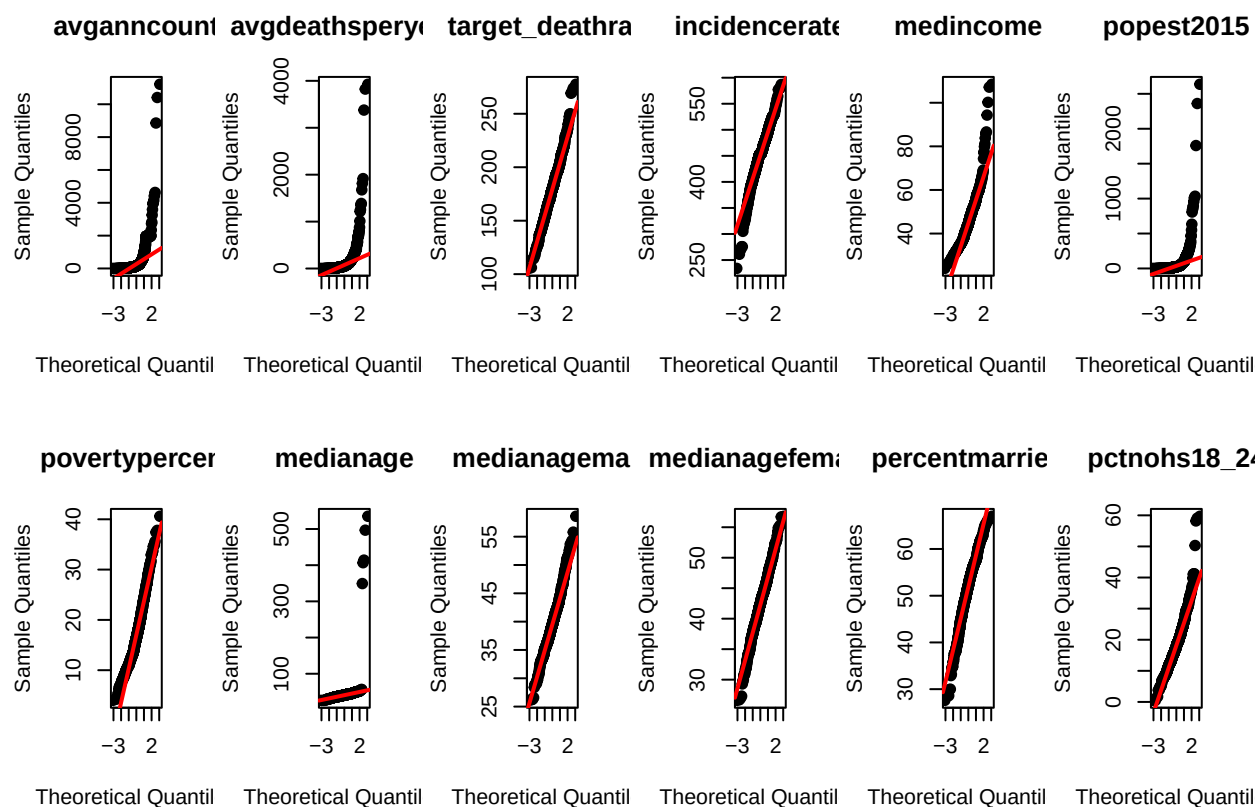


```
par(mfrow=c(3,6))
for (i in 13:27)
{
  x=data[,i]
  hist(x, freq=FALSE, main=colnames(data)[i])
  mn=mean(x, na.rm=TRUE)
  std=sd(x, na.rm=TRUE)
  curve(dnorm(x, mean=mn, sd=std), col='red', lwd=2, add=TRUE)
}
```



Now we can plot the QQ-plots:

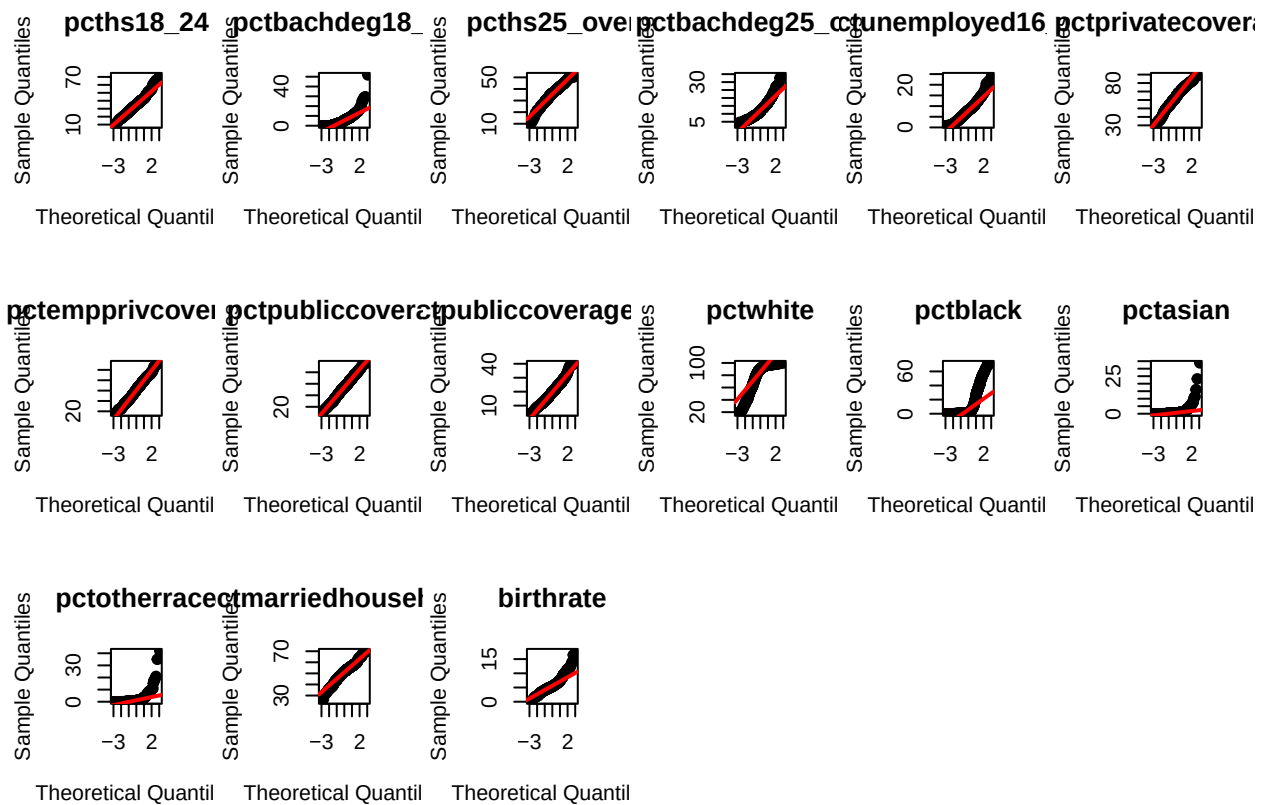
```
par(mfrow=c(2,6))
for (i in 1:12){
  qqnorm(data[,i], pch=19 , main=colnames(data)[i])
  qqline(data[,i], col='red', lwd=2)
}
```



```

par(mfrow=c(3,6))
for (i in 13:27){
  qqnorm(data[,i], pch=19 , main=colnames(data)[i])
  qqline(data[,i], col='red', lwd=2)
}

```



Some of them seems to follow a normal distribution just like “pctpubliccoverage”, conversely others do not follow a normal distribution like “pctblack”. We can check this by both looking at the histograms and also at the qnorm plots. To have additional evidence we can also calculate the Shapiro-Wilk value:

```
for (i in 1:27){
  print(shapiro.test(data[,i]))
}

##
##  Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.45698, p-value < 2.2e-16
##
##
##  Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.34418, p-value < 2.2e-16
##
##
##  Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.99289, p-value = 0.01801
##
##
```

```

## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.97673, p-value = 3.774e-07
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.91129, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.30966, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.96487, p-value = 1.458e-09
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.14726, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.99061, p-value = 0.002837
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.99521, p-value = 0.1262
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.97959, p-value = 1.823e-06
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.94338, p-value = 7.009e-13
##
##

```

```

## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.98847, p-value = 0.0005595
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.82584, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.98415, p-value = 2.867e-05
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.93303, p-value = 3.509e-14
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.97091, p-value = 2.088e-08
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.98855, p-value = 0.0005958
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.9964, p-value = 0.3224
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.99784, p-value = 0.7802
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.98884, p-value = 0.0007389
##
##

```

```
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.80129, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.6611, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.3726, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.49672, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.98205, p-value = 7.718e-06
##
##
## Shapiro-Wilk normality test
##
## data:  data[, i]
## W = 0.92394, p-value = 3.265e-15
```

We have checked that only two variables are normal: “pctpubliccovarage” and “pctempprivcov”.

We now study if some values are missing:

```
colSums(is.na(data))
```

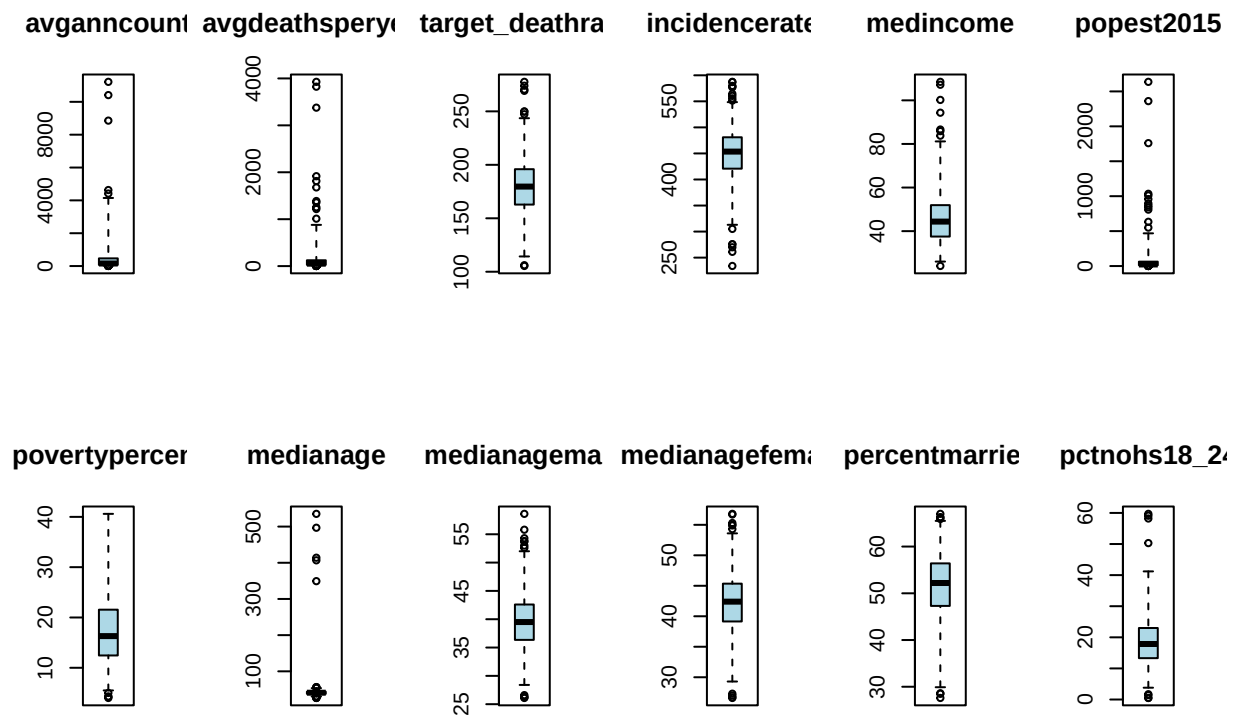
```
##          avganncount      avgdeathsperyear      target_deathrate
##                0                0                0
##      incidencerate      medincome      popest2015
##                0                0                0
##      povertypercent      medianage      medianagemale
##                0                0                0
##      medianagefemale      percentmarried      pctnohs18_24
##                0                0                0
##      pcths18_24      pctbachdeg18_24      pcths25_over
##                0                0                0
##      pctbachdeg25_over      pctunemployed16_over      pctprivatecoverage
##                0                0                0
##      pctempprivcoverage      pctpubliccoverage      pctpubliccoveragealone
```

```
##          0          0          0
##          pctwhite          pctblack          pctasian
##          0          0          0
##          pctotherrace  pctmarriedhouseholds  birthrate
##          0          0          0
```

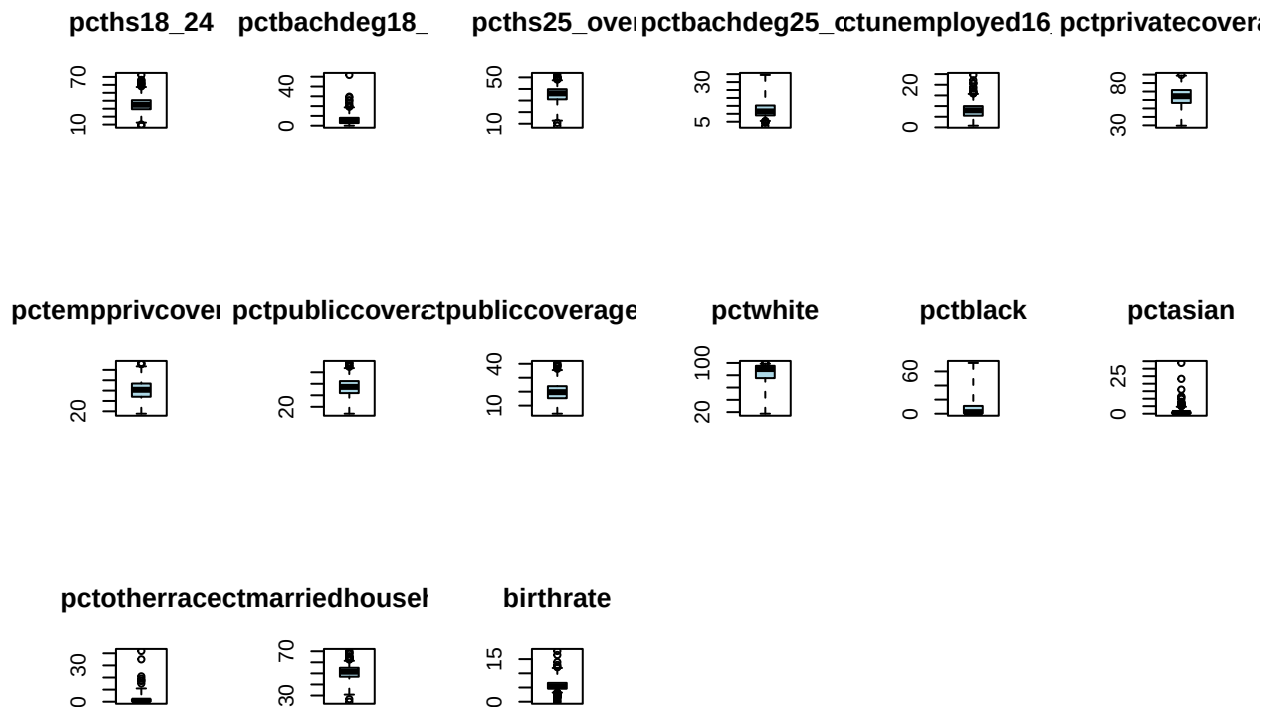
None of the values are missing!!

We now check if we have some outliers:

```
par(mfrow= c(2,6))
for (i in 1:12){
  adjbox(data[,i], main=colnames(data)[i], col='lightblue')
}
```



```
par(mfrow= c(3,6))
for (i in 13:27){
  adjbox(data[,i], main=colnames(data)[i], col='lightblue')
}
```



We can see that some of the variables have outliers.

We transform the strongly skewed variables ($\text{skewness} < -1$ and $\text{skewness} > 1$) to get a skewness closer to 0. We decide to use the function `transfo` as explained in the paper to make our calculation easier and more efficient. In this case we are using the YJ transformation, but it's important to note that we tried also with the `boxcox`, the issue was that it was not possible to apply because of the presence of values ≤ 0 . Here you can find a proof that it is not possible to apply the box-cox transformation:

```
checkboxcox=function(dataset){
  invalidcolumn=names(dataset)[sapply(dataset, function(x) any(x<=0, na.rm=TRUE))]
  if (length(invalidcolumn)>0){
    cat("We can't use Box-Cox directly\n")
    cat("Invalid columns: ")
    print(invalidcolumn)
  }
  else {
    print("I can use Box-Cox")
  }
}
checkboxcox(data)
```

```
## We can't use Box-Cox directly
## Invalid columns: [1] "pctbachdeg18_24" "pctblack" "pctasian" "pctotherrace"
## [5] "birthrate"
```

If we wanted we could sum a small constant like 0.1, 0.01 or 0.001 to be able to apply the `boxcox` but in this case it is more efficient to directly apply the YJ transformation to the whole dataset. Indeed another issue with adding a small constant is that if I have a lot of zeros adding it moves the distribution.

We now reconstruct the dataset with the transformed variables applying `trasfo` since it is able to see which variables need transformation and which not. It is quite important to underline that we isolate the covariates to apply DDC algorithm only to the explanatory variables, in this case the variable “target_deathrate” must not be imputed to prevent data leakage:

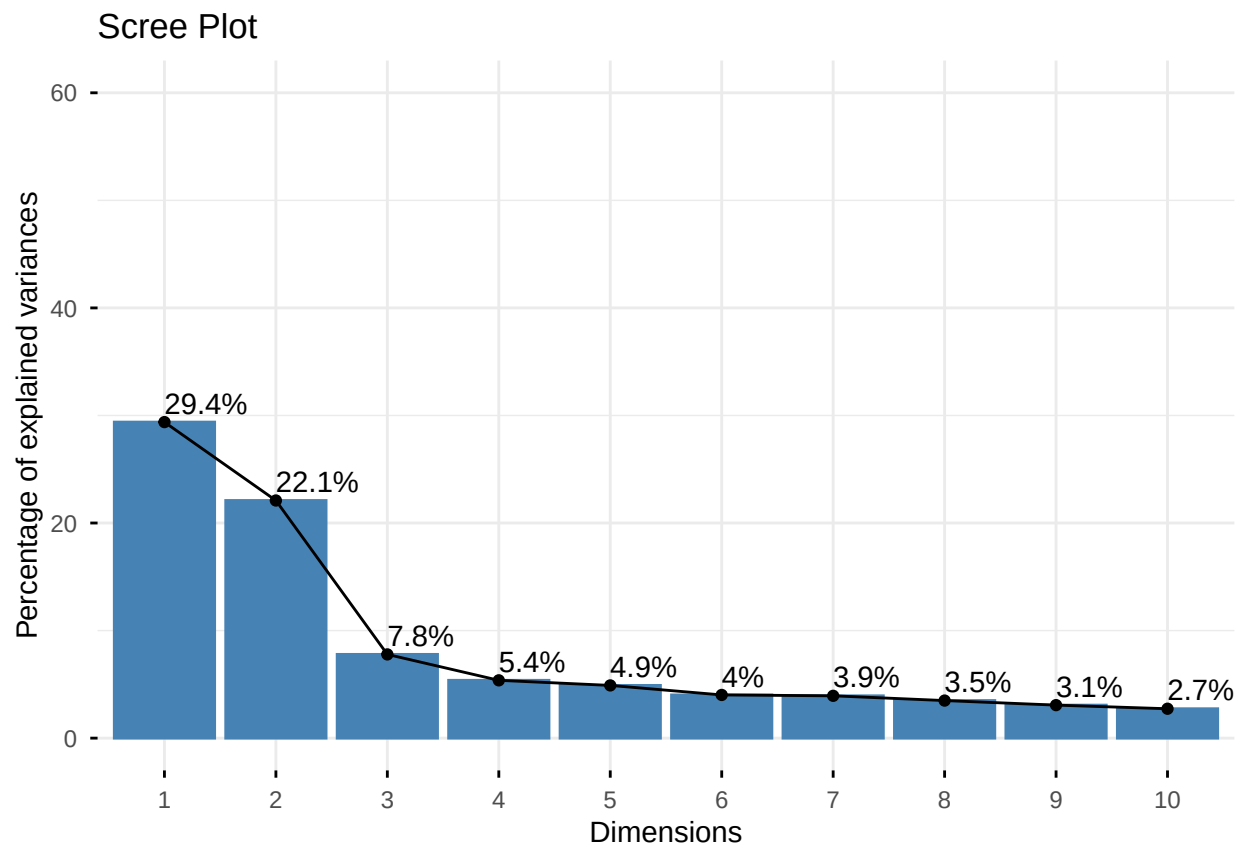
```
xcovariate=numdata[,colnames(numdata)!='target_deathrate']
xmatrix=as.matrix(xcovariate)
transform=transfo(xmatrix, type='YJ', robust=TRUE)
```

```
##
## The input data has 500 rows and 26 columns.
```

```
xtrans=transform$Y
```

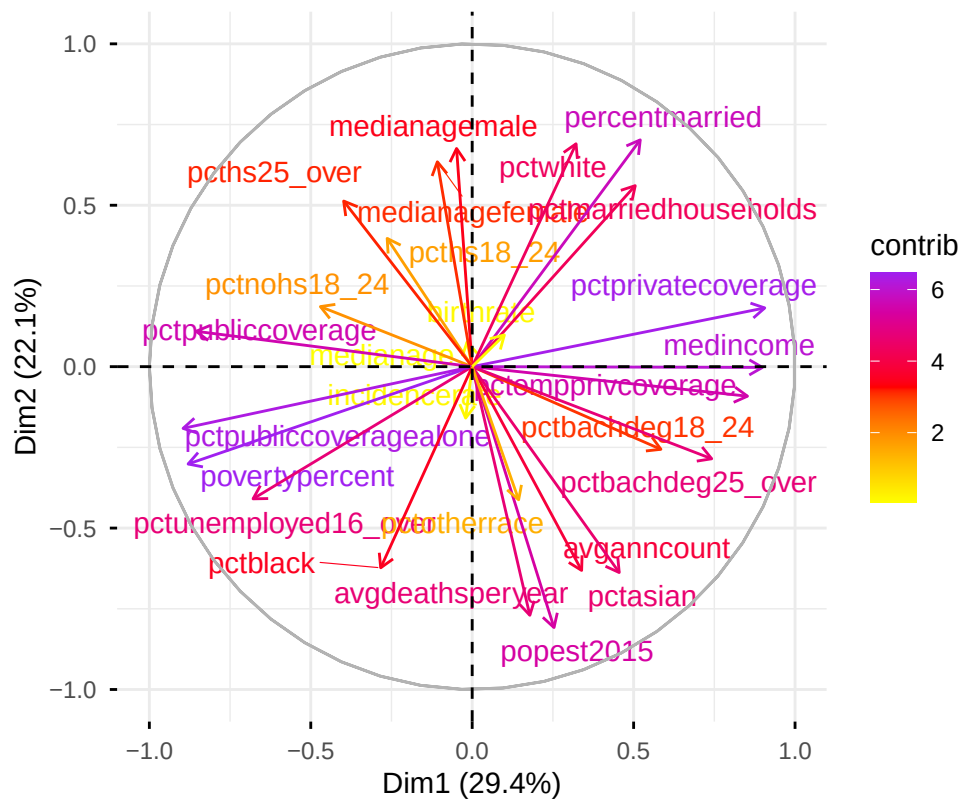
We can use PCA to see what happens:

```
pcaout=prcomp(xtrans, scale. = TRUE)
fviz_eig(pcaout, addlabels = TRUE, ylim=c(0,60), barfill = 'steelblue', barcolor='steelblue', main='Scree Plot')
```



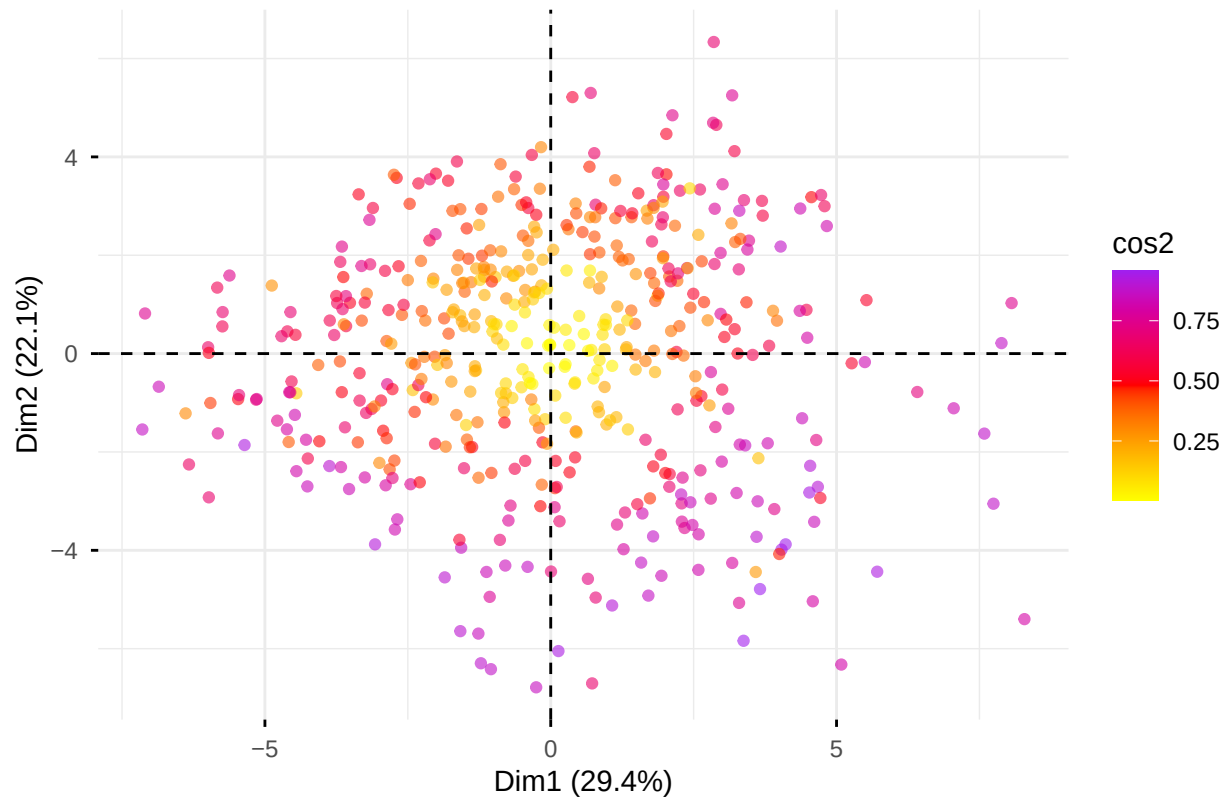
```
fviz_pca_var(pcaout, repel=TRUE, gradient.cols=c('yellow', 'red', 'purple'), col.var='contrib', title='Map of variables on the first two principal components')
```

Map PCA variable



```
fviz_pca_ind(pcaout, col.ind='cos2', gradient.cols=c('yellow', 'red', 'purple'),geom='point', alpha.ind
```

Map PCA Counties



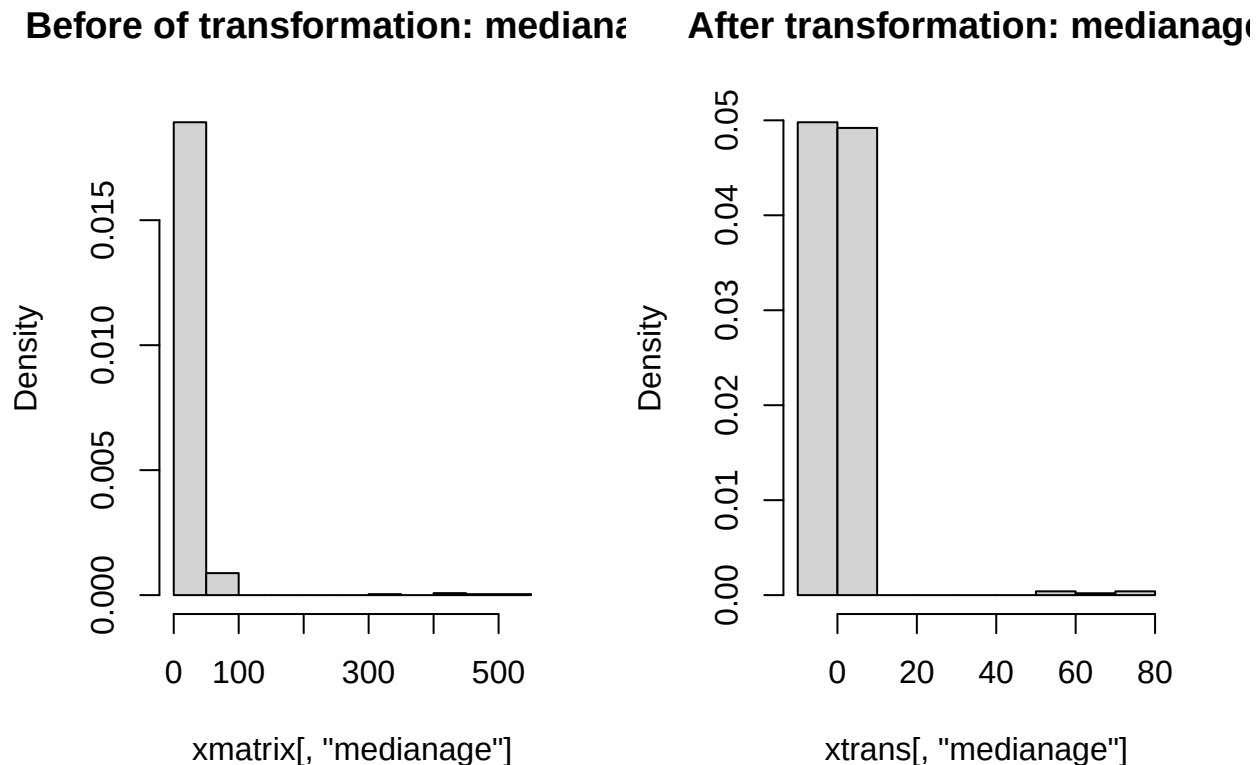
The PCA confirms multicollinearity, indeed an evidence are the tightly clustered variables vectors. The PCA map is characterized by a dispersed data cloud with some observations lying far from the central mass, suggesting the presence of outliers. In addition, the screeplot shows that the first two dimensions capture around the 50% of total variance.

To show that our transformation is good we calculate skewness:

```
skewness(xtrans)
```

##	avganncount	avgdeathsperyear	incidencerate
##	0.259136868	0.178056038	-0.079569278
##	medincome	popest2015	povertypercent
##	0.617894903	0.218899126	-0.056248224
##	medianage	medianagemale	medianagefemale
##	9.775545168	-0.075896651	0.029279443
##	percentmarried	pctnohs18_24	pcths18_24
##	-0.099731100	0.306129911	0.258015457
##	pctbachdeg18_24	pcths25_over	pctbachdeg25_over
##	0.482314175	-0.162891248	0.068902660
##	pctunemployed16_over	pctprivatecoverage	pctempprivcoverage
##	0.380672071	0.007226597	0.098366239
##	pctpubliccoverage	pctpubliccoveragealone	pctwhite
##	0.060138982	0.195321221	-0.184291824
##	pctblack	pctasian	pctotherrace
##	0.327132774	0.174947895	0.208024814
##	pctmarriedhouseholds	birthrate	
##	0.674239509	-0.619907503	

```
#we can also compare the histograms before and after:
par(mfrow=c(1,2))
hist(xmatrix[, "medianage"], main='Before of transformation: medianage', freq=FALSE)
hist(xtrans[, "medianage"], main='After transformation: medianage', freq=FALSE)
```



The transformation was effective, indeed only one variable has $|\text{skewness}| > 1$. “medianage” has a right skewness of 9.77. However, as we have explained previously it has some extreme wrong entry data. In addition, if we look at the two histograms we can see that the transformation changes the scale, but on the other hand we are still able to find anomalies, so that they don’t get masked by the transformation. As a consequence a statistical transformation can’t and should not mask this behavior, making our transformation satisfying.

We can also check if the normality has improved through Shapiro-Wilk test:

```
for (i in 1:ncol(xtrans)) {
  print(shapiro.test(xtrans[,i]))
}
```

```
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.95495, p-value = 3.191e-11
##
##
## Shapiro-Wilk normality test
```

```

##
## data:  xtrans[, i]
## W = 0.97825, p-value = 8.587e-07
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.9901, p-value = 0.001915
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.97377, p-value = 8.272e-08
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.97206, p-value = 3.61e-08
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99485, p-value = 0.09363
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.18023, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99584, p-value = 0.2101
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99645, p-value = 0.334
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99656, p-value = 0.3636
##
##
## Shapiro-Wilk normality test

```

```

##
## data:  xtrans[, i]
## W = 0.98795, p-value = 0.0003844
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99218, p-value = 0.01002
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.97611, p-value = 2.725e-07
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99629, p-value = 0.2965
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99654, p-value = 0.3588
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.98715, p-value = 0.0002174
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99683, p-value = 0.4386
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99603, p-value = 0.2443
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.99776, p-value = 0.7514
##
##
## Shapiro-Wilk normality test

```

```

##
## data:  xtrans[, i]
## W = 0.99481, p-value = 0.09042
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.95316, p-value = 1.7e-11
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.90955, p-value < 2.2e-16
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.98071, p-value = 3.467e-06
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.96305, p-value = 6.889e-10
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.97186, p-value = 3.281e-08
##
##
## Shapiro-Wilk normality test
##
## data:  xtrans[, i]
## W = 0.96492, p-value = 1.489e-09

```

Through the Shapiro-Wilk we can see that now several variables pass the normality test ($p > 0.05$). It's important to notice that the DDC algorithm performs optimally under Gaussian assumption, but strict normality is not a prerequisite. Indeed it just requires the absence of many heavy skewness, in order to avoid false positive outlier detection. Since now all the covariates satisfy this assumption (except for “medianage” but it is a problem in the dataset itself) we can apply the DDC algorithm. (Note: in our case it does not make sense to eliminate these data entry errors because the goal is to show that the model is able to detect them).

We set the tuning parameter by setting ‘tolProb=0.99’ through DDCpars list. In this way we defined the tolerance probability threshold for identifying a cell as an outlier. We chose 0.99 as a parameter, to ensure that the DDC works only on actual outliers, preserving original data.

```

outddc=DDC(xtrans, DDCpars=list(tolProb=0.99))

```

```

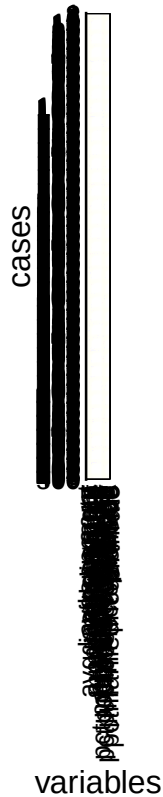
##

```

```
## The input data has 500 rows and 26 columns.
```

```
cellMap(outddc$stdResid, columnlabels=colnames(xtrans), rowlabels=as.character(1:nrow(xtrans)))
```

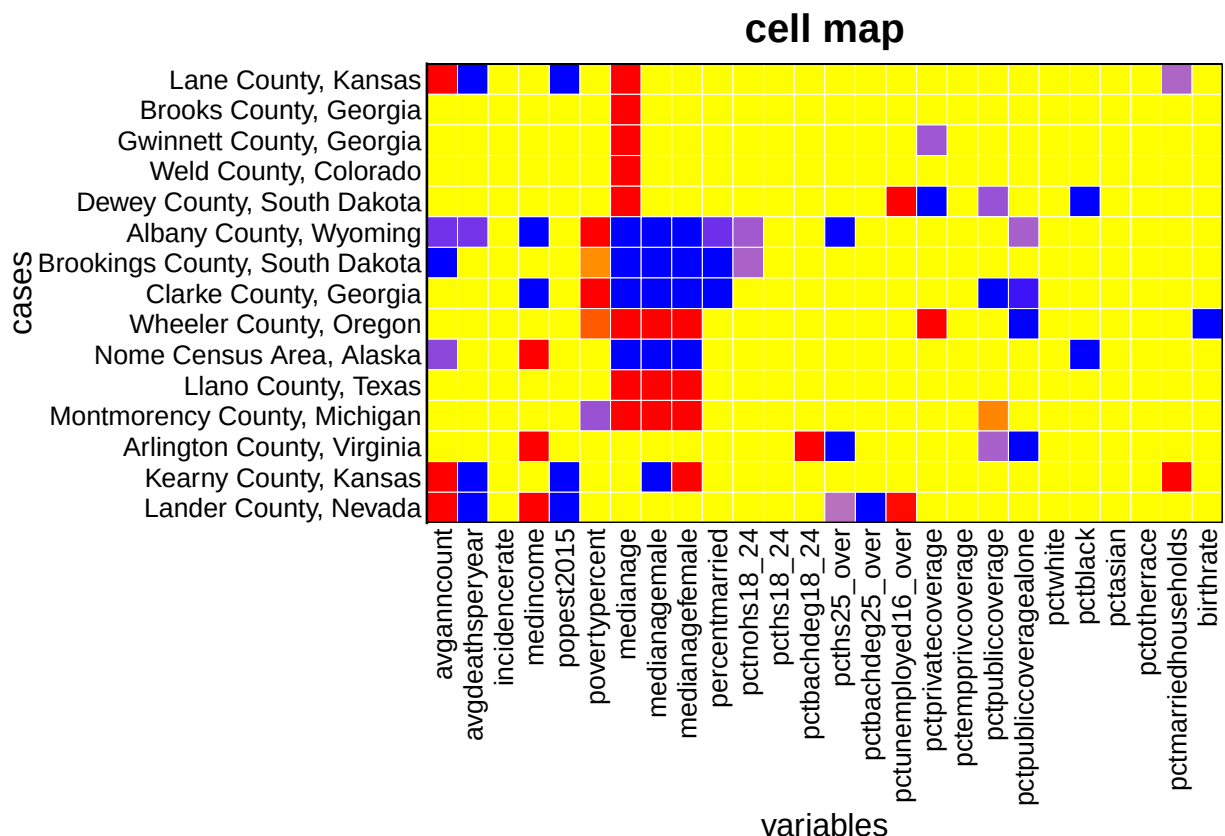
cell map



```
ximputed=outddc$Ximp
```

We can't have a good representation so we try again in a different way, we detect anomalies and then choose the counties with the highest anomalies, We start with the standardize residuals created with DDC, so based on their value we can understand if they are outliers or not. Then we consider the absolute value and then sum those values for each rows. After we consider the first fifteen counties with the highest rowsum.

```
anomaly=rowSums(abs(outddc$stdResid), na.rm=TRUE)
worstindex=order(anomaly, decreasing=TRUE)[1:15]
worstresiduals=outddc$stdResid[worstindex, ]
cellMap(worstresiduals, columnlabels = colnames(xtrans), rowlabels = originalfull$geography[rsub[worstindex, ]])
```



Yellow=the value that the algorithm was predicting Red=the value is way higher than expected Orange=the value is higher than expected (but not too much) Blue=the value is way smaller than expected Purple=the value is smaller than expected (but not too much) White=missing data (that in this case we don't have)

We can clearly see cell outliers, so based on the county name we can do an analysis. We also consider the row wise outliers to compare them with the cellwise ones:

```
celloutliers=sum(abs(outddc$stdResid)>2.56, na.rm=TRUE)
totalcell=nrow(xtrans)*ncol(xtrans)
perccelloutliers=(celloutliers/totalcell)*100
cat("Anomalous cell: ", perccelloutliers, "%\n")
```

```
## Anomalous cell: 4.146154 %
```

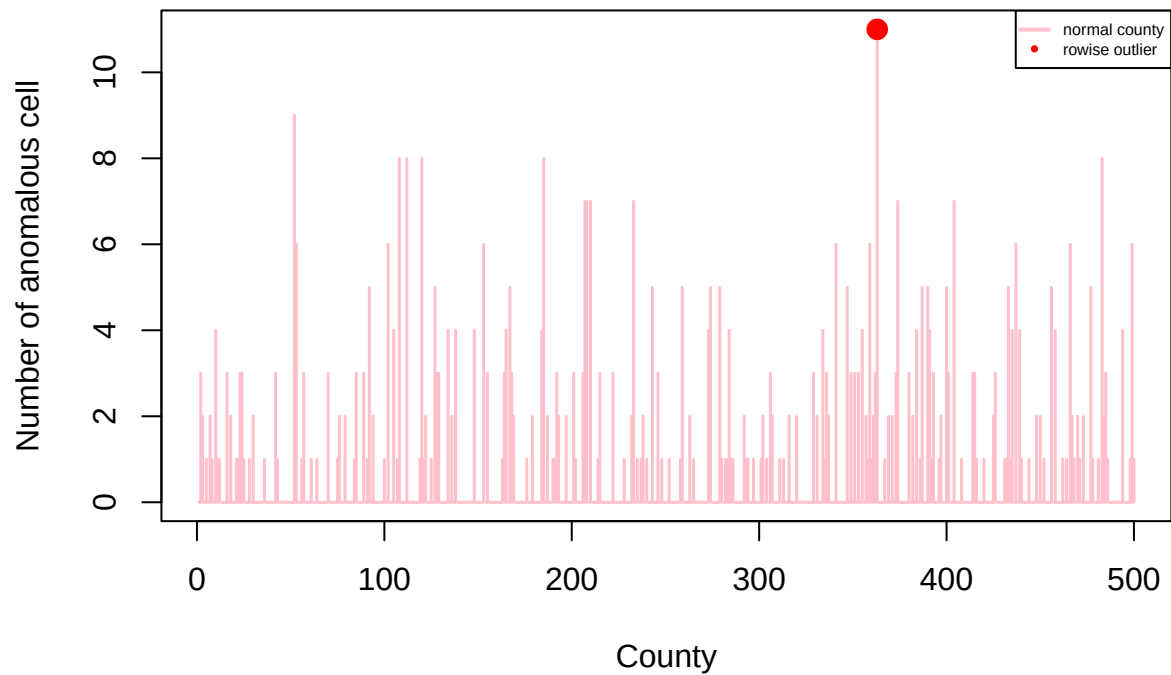
```
rowoutliers=outddc$indrows
cat("Number outliers per row: ", length(rowoutliers), "\n")
```

```
## Number outliers per row: 1
```

Now we consider a row wise plot:

```
anomaliesperrow=rowSums(abs(outddc$stdResid)>2.56, na.rm=TRUE)
plot(anomaliesperrow, type='h', col='pink', lwd=1.5, main='Rowwise outliers', xlab='County', ylab='Number')
points(rowoutliers, anomaliesperrow[rowoutliers], col='red', pch=16, cex=1.5)
legend('topright', legend=c('normal county', 'rowwise outlier'), col=c('pink','red'), pch=c(NA, 16), lty=
```

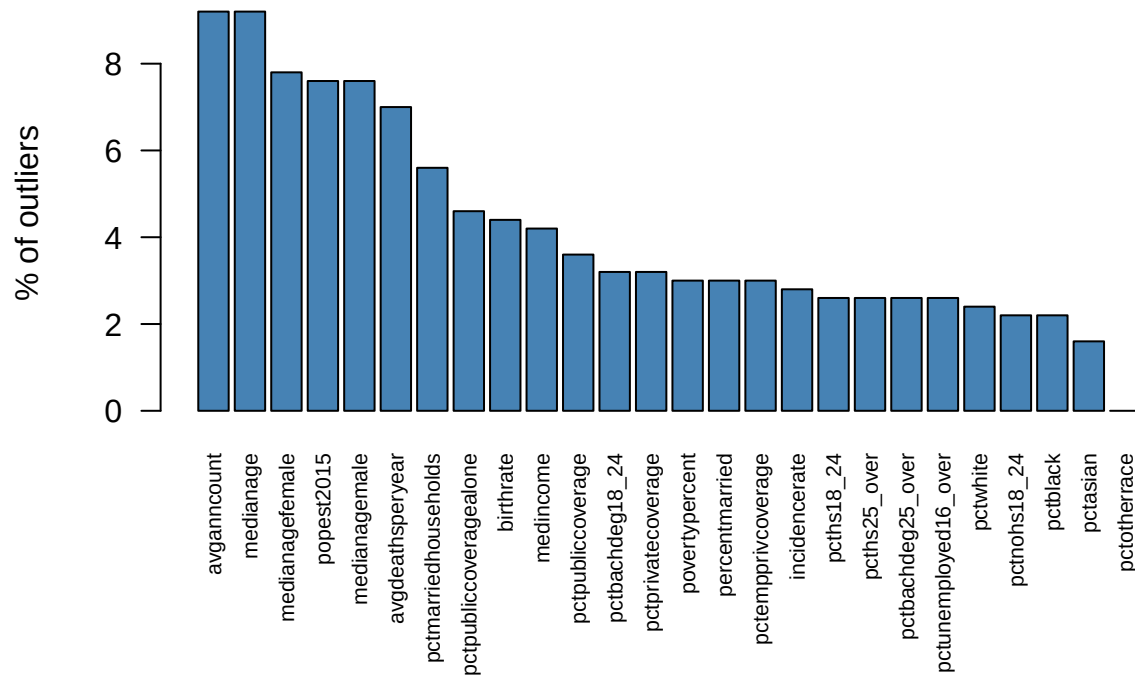
Rowwise outliers



We can show how many anomalies we have for each variable:

```
colanomalies=colSums(abs(outddc$stdResid)>2.56, na.rm=TRUE)/nrow(xtrans)*100
par(mar=c(8,4,4,2))
barplot(sort(colanomalies, decreasing=TRUE), las=2, col='steelblue', main='Percentage outliers per vari
```

Percentage outliers per variable



```
source("cellLTS_code.R")
```

We study the influence of these outliers on the regression:

```
target=data$target_deathrate
dataraw=data.frame(target_deathrate=target, xtrans)
dataddc=data.frame(target_deathrate=target, ximputed)
```

We can now train the linear regression model on both the clean dataset and the dirty one. Before of doing so we have to choose the hyperparameters to balance the trade off between robustness and efficiency. We decided to use setting='KS2014' because it is the most suitable one when having multiple predictors (Koller & Stahel). In this way the loss functions achieve the maximum breakdown point of 50% while maintaining an efficiency of 50% if errors were normally distributed.

We can study multicollinearity, for $vif > 10$ we have multicollinearity and for $vif > 30$ we have strong multicollinearity. Indeed severe multicollinearity inflates the variance of the estimated coefficients leading to unstable p-value, for this reason we decided to perform a backward elimination. At each step the highest VIF(>10) was removed. This allowed a more stable and interpretable model.

```
classicmodel = lm(target_deathrate ~ ., data = dataraw)
vifval=vif(classicmodel)
print('Initial vif: ')
```

```
## [1] "Initial vif: "
```

```
print(vifval)
```

```
##          avganncount      avgdeathsperyear      incidencerate
##          3.335720          82.157072          1.371453
##          medincome        popest2015        povertypercent
##          10.555404          90.084778          12.125692
##          medianage        medianagemale      medianagefemale
##          1.081312          8.814289          10.470860
##          percentmarried    pctnohs18_24      pcths18_24
##          8.061788          1.798890          1.852166
##          pctbachdeg18_24    pcths25_over      pctbachdeg25_over
##          1.962147          3.204641          4.392551
##          pctunemployed16_over  pctprivatecoverage  pctempprivcoverage
##          2.856206          16.963337          8.610205
##          pctpubliccoverage  pctpubliccoveragealone  pctwhite
##          25.984675          25.471324          5.571641
##          pctblack          pctasian          pctotherrace
##          3.728633          2.538461          1.685243
##          pctmarriedhouseholds  birthrate
##          5.368886          1.322164
```

```
#we can see that some of them have vif larger than 10 so we try to eliminate popest2015 as the first on
classicmodel2=lm(target_deathrate~ . - popest2015, data = dataraw)
vif2=vif(classicmodel2)
print('new vif2: ')
```

```
## [1] "new vif2: "
```

```
print(vif2) #now avgdeathsperyear has vif<10
```

```
##          avganncount      avgdeathsperyear      incidencerate
##          3.334213          4.401989          1.255281
##          medincome        povertypercent      medianage
##          10.410758          12.109435          1.079946
##          medianagemale      medianagefemale      percentmarried
##          8.772254          10.148321          7.612894
##          pctnohs18_24      pcths18_24      pctbachdeg18_24
##          1.798619          1.829401          1.961764
##          pcths25_over      pctbachdeg25_over  pctunemployed16_over
##          3.111487          4.392548          2.852907
##          pctprivatecoverage  pctempprivcoverage  pctpubliccoverage
##          16.769347          8.564088          25.245733
##          pctpubliccoveragealone  pctwhite      pctblack
##          25.177922          5.568947          3.726279
##          pctasian          pctotherrace  pctmarriedhouseholds
##          2.510605          1.685017          5.017155
##          birthrate
##          1.302414
```

```
classicmodel3=lm(target_deathrate~ . - popest2015- pctpubliccoverage, data = dataraw)
vif3=vif(classicmodel3)
print('new vif3: ')
```

```
## [1] "new vif3: "
```

```
print(vif3) #now pctpubliccoveragealone has vif<10
```

##	avganncount	avgdeathsperyear	incidencerate
##	3.328166	4.324050	1.255079
##	medincome	povertypercent	medianage
##	10.205938	12.109306	1.065821
##	medianagemale	medianagefemale	percentmarried
##	8.647412	8.568266	7.590108
##	pctnohs18_24	pcths18_24	pctbachdeg18_24
##	1.794390	1.816938	1.956597
##	pcths25_over	pctbachdeg25_over	pctunemployed16_over
##	3.090610	4.182555	2.768549
##	pctprivatecoverage	pctempprivcoverage	pctpubliccoveragealone
##	12.654042	6.452847	7.109096
##	pctwhite	pctblack	pctasian
##	5.535457	3.697992	2.509236
##	pctotherrace	pctmarriedhouseholds	birthrate
##	1.683614	5.009931	1.299701

```
classicmodel4=lm(target_deathrate~ . - popest2015- pctpubliccoverage- medincome, data = dataraw)  
vif4=vif(classicmodel4)  
print('new vif4: ')
```

```
## [1] "new vif4: "
```

```
print(vif4)
```

##	avganncount	avgdeathsperyear	incidencerate
##	3.315739	4.276648	1.254986
##	povertypercent	medianage	medianagemale
##	6.110325	1.065773	8.632674
##	medianagefemale	percentmarried	pctnohs18_24
##	8.501901	7.359299	1.794099
##	pcths18_24	pctbachdeg18_24	pcths25_over
##	1.784308	1.952769	2.949059
##	pctbachdeg25_over	pctunemployed16_over	pctprivatecoverage
##	4.153247	2.766735	12.648609
##	pctempprivcoverage	pctpubliccoveragealone	pctwhite
##	6.175278	7.096202	5.223436
##	pctblack	pctasian	pctotherrace
##	3.653160	2.505218	1.683547
##	pctmarriedhouseholds	birthrate	
##	4.702964	1.299635	

```
classicmodel5=lm(target_deathrate~ . - popest2015- pctpubliccoverage- medincome-pctprivatecoverage, data = dataraw)  
vif5=vif(classicmodel5)  
print('new vif5: ')
```

```
## [1] "new vif5: "
```

```
print(vif5)
```

```
##          avganncount      avgdeathsperyear      incidencerate
##          3.233561          4.175907          1.237820
##          povertypercent      medianage      medianagemale
##          5.955405          1.062789          8.616202
##          medianagefemale      percentmarried      pctnohs18_24
##          8.018920          7.358673          1.735464
##          pcths18_24      pctbachdeg18_24      pcths25_over
##          1.717547          1.899584          2.925149
##          pctbachdeg25_over      pctunemployed16_over      pctempprivcoverage
##          4.020818          2.765429          4.722183
##          pctpubliccoveragealone      pctwhite      pctblack
##          5.179111          5.168032          3.638868
##          pctasian      pctotherrace      pctmarriedhouseholds
##          2.505188          1.664279          4.695785
##          birthrate
##          1.276586
```

IMPORTANT CAVEAT: we stopped the elimination at four variables to preserve the predictive power of the model (we tried with removing more variables but the results were not good), in this way we decided to accept some residuals multicollinearity to better highlight the masking effect in a higher-dimensional space.

We only consider the new number of reduced variables:

```
varrem = c("popest2015", "pctpubliccoverage", "medincome", "pctprivatecoverage")
datarawfinal=dataraw[, !(names(dataraw)%in% varrem)]
dataddcfinal=dataddc[, !(names(dataddc) %in% varrem)]
xtransfinal=xtrans[, !(colnames(xtrans) %in% varrem)]
classicmodelfinal=lm(target_deathrate~.,data=datarawfinal )
modelcleanMMfinal=lmrob(target_deathrate~.,data=dataddcfinal, control = lmrob.control(setting="KS2014",
cellLTsmodfinal=cellLTs(xtransfinal, target)
```

```
coeffclassic=coef(classicmodelfinal)
coeffrobddc=coef(modelcleanMMfinal)
intercept1=cellLTsmodfinal$intercept
coefficients1=cellLTsmodfinal$coefficients
coeffcellts=c(intercept1, coefficients1)
tablecompare=data.frame(
  ClassicRaw=as.numeric(coeffclassic),
  RobustDDC=as.numeric(coeffrobddc),
  CellLTs=as.numeric(coeffcellts)
)
tablecompare=round(tablecompare, 3)
rownames(tablecompare) = c("Intercept", colnames(xtransfinal))
print(tablecompare)
```

```
##          ClassicRaw RobustDDC CellLTs
## Intercept          180.680    154.328 181.210
## avganncount         -4.338   -152.096   8.737
## avgdeathsperyear      8.600    140.790  -1.900
## incidencerate         9.014     14.400   6.458
```

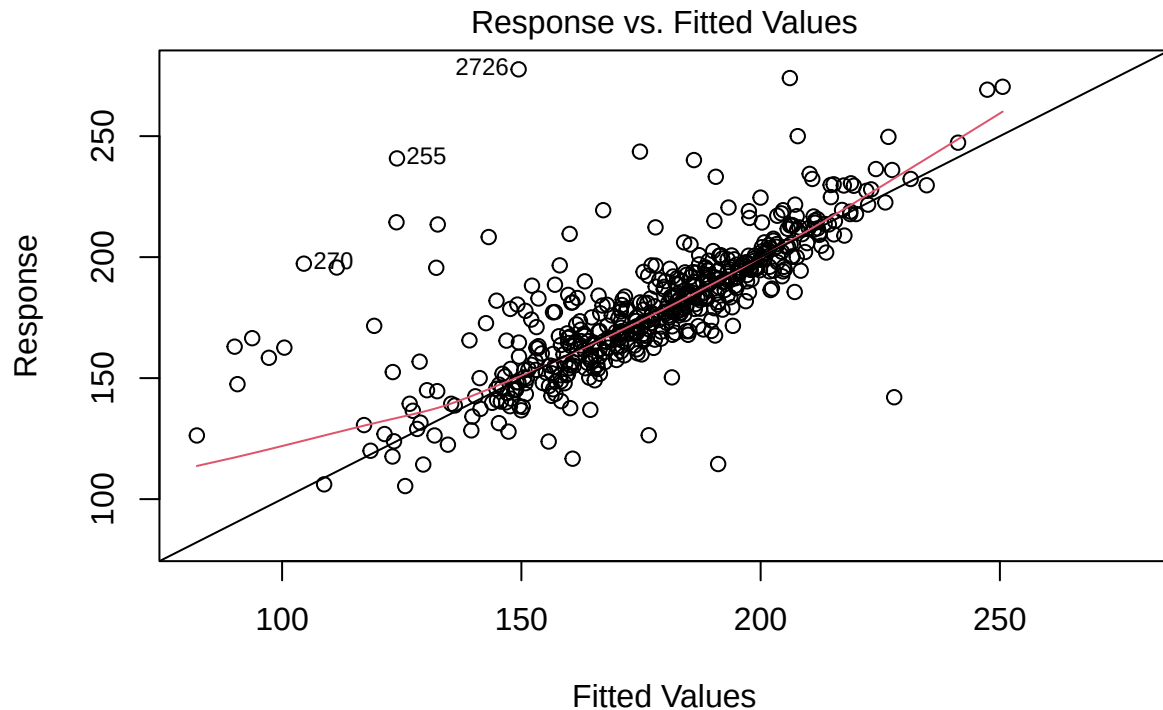
## povertypercent	4.298	3.946	5.975
## medianage	-0.196	-5.058	33.621
## medianagemale	1.169	4.468	-19.586
## medianagefemale	-2.607	-1.262	-16.271
## percentmarried	4.373	5.148	4.917
## pctnohs18_24	1.621	0.635	1.130
## pcths18_24	0.875	1.603	0.404
## pctbachdeg18_24	-1.075	0.091	-0.530
## pcths25_over	0.874	-1.659	1.117
## pctbachdeg25_over	-3.984	-1.509	-8.678
## pctunemployed16_over	-0.330	1.814	-0.649
## pctempprivcoverage	1.567	3.551	1.675
## pctpubliccoveragealone	4.132	-0.089	1.976
## pctwhite	0.530	-3.606	1.019
## pctblack	1.230	-1.284	2.993
## pctasian	-2.812	-0.650	-3.067
## pctotherrace	-3.391	-2.735	-3.180
## pctmarriedhouseholds	-4.713	-3.673	-3.626
## birthrate	0.686	1.137	-0.789

First of all, we can see the masking effect because of the extreme values. An example is “pctwhite”, indeed with the OLS estimator we got 0.530, while with DDC we got -3.606. In reality, white people have a lower probability of dying because of different socio-economic reasons, so the robust DDC was the only one able to spot this behavior. Then we can also see a scale distortion, if we look at “avgdeathperyear” and “avgnncount” we can find high value like |-152| and 140 compared to the cellLTS which are closer to zero. This probably happens because the variables are still dominated by some counties with extremely large values. At the end we can also see a good behavior of cellLTS with variable with entry error, infact it was able to trim the cells with extremely weird values (like 400 as an age).

We can plot modelcleanMMfinal:

```
plot(modelcleanMMfinal, which=3, main='Outlier Map:Robust Distance vs Residuals')
```

Outlier Map: Robust Distance vs Residuals



```
lm(formula = target_deathrate ~ ., data = dataddcfinal, control = lmrob.control(setting = "k", handle = 500))
```

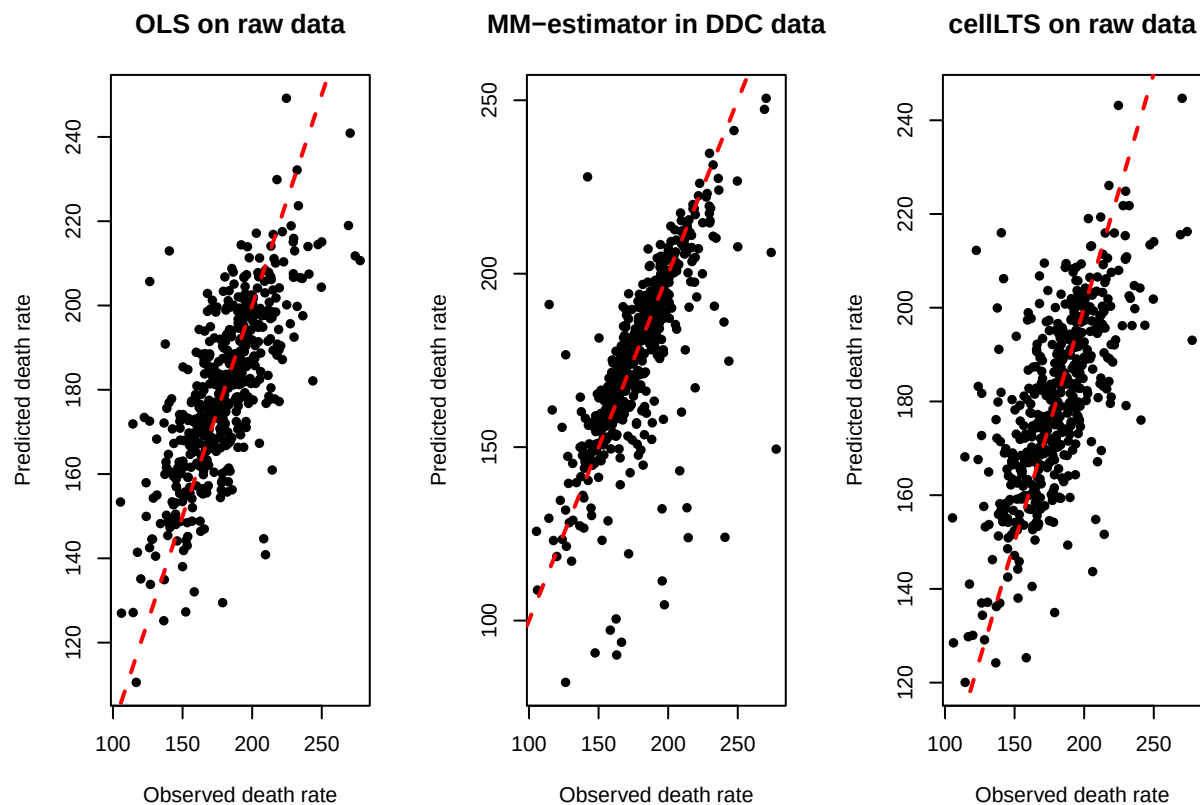
We now plot observed vs predicted:

```
par(mfrow=c(1,3))
plot(datarawfinal$target_deathrate, fitted(classicmodelfinal), xlab='Observed death rate', ylab='Predicted death rate',
     abline(0,1, col='red', lwd=2, lty=2))

plot(dataddcfinal$target_deathrate, fitted(modelcleanMMfinal), xlab='Observed death rate', ylab='Predicted death rate',
     abline(0,1, col='red', lwd=2, lty=2))

previsioncellLTS = as.matrix(cbind(1, cellLTSmodfinal$Ximp)) %*% cellLTSmodfinal$coefficients

plot(datarawfinal$target_deathrate, previsioncellLTS, xlab = 'Observed death rate', ylab = 'Predicted death rate',
     abline(0, 1, col = 'red', lwd = 2, lty = 2))
```



The scatterplots compare the performances of the three models, the OLS shows high dispersion distorted by cellwise outliers, in the meanwhile also the CellLTS is an improvement compared to OLS but still more distorted than MM-estimator. We can conclude that the visually best one is the MM-estimator applied to the DDC imputed data thanks to the reconstruction of erroneous cells.

```
weightsmm=modelcleanMMfinal$rweights
obsnoweight=sum(weightsmm<0.5)
cat(obsnoweight)
```

```
## 40
```

```
worstcounty=which.min(weightsmm)
```

This means that around the 8% of counties have so strong outliers or leverage points that the algorithm has to mitigate their effect.

We can also calculate the weights of cellLTS

```
weightcellLTS=cellLTSmodfinal$W
disregardedcells=sum(weightcellLTS==0)
cat('Disregarded cell with cellLTS: ', disregardedcells, '\n')
```

```
## Disregarded cell with cellLTS: 580
```

The algorithm disregarded 580 cells, since we have around 11.000 total cells (22 variable*500 counties), this implies that around 5% of data are anomalous. This shows how much essential robust estimates are in this case.

We can also compare the residuals scales just like they did in the paper:

```
madclassic=mad(resid(classicmodelfinal))
madddc=mad(resid(modelcleanMMfinal))
madcellLts=mad(resid(cellLTSmodfinal))
```

```
cat('MAD: \n')
```

```
## MAD:
```

```
cat('OLS: ', madclassic)
```

```
## OLS: 15.97971
```

```
cat('\nMM-estimator: ', madddc)
```

```
##
```

```
## MM-estimator: 9.239968
```

```
cat('\nCellLTS: ', madcellLts)
```

```
##
```

```
## CellLTS: 15.62573
```

We can see that MM-estimator had the best MAD since smaller, while the worst one is the OLS one.

```
sumclassic=summary(classicmodelfinal)
sumclean=summary(modelcleanMMfinal)
madtarget = mad(datarawfinal$target_deathrate)
RcellLTS=1-(madcellLts/madtarget)^2
cat('R-squared classic model: ', sumclassic$r.squared)
```

```
## R-squared classic model: 0.5293018
```

```
cat('\nR-squared robust MM: ', sumclean$r.squared)
```

```
##
```

```
## R-squared robust MM: 0.832272
```

```
cat('\nR-squared cellLTS: ', RcellLTS)
```

```
##
```

```
## R-squared cellLTS: 0.5870046
```

```
pvalclassic=sumclassic$coefficients[,4]
pvalclean=sumclean$coefficients[,4]
signigclassic=names(pvalclassic[pvalclassic<0.05])
signigclean=names(pvalclean[pvalclean<0.05])
cat('\nSignificative variable in classic but not MM robust: \n')
```

```
##
## Significant variable in classic but not MM robust:

print(setdiff(signigclassic, signclean))

## [1] "pctbachdeg25_over"      "pctpubliccoveragealone" "pctasian"

cat('\nSignificant variable in MM robust but not in classic: \n')

##
## Significant variable in MM robust but not in classic:

print(setdiff(signclean, signigclassic))

## [1] "pcths18_24"      "pcths25_over"      "pctunemployed16_over"
## [4] "pctempprivcoverage" "pctwhite"          "birthrate"
```

We can see the effects of masking effect caused by data contamination. First of all, the goodness of fit (R^2) improves, increasing from 0.529 in the OLS to 0.832 in the MM estimator. This implies that the robust model is able to explain a larger portion of the variance in the target death rate. In addition, we also found a R^2 of 0.587 for cellLTS which is theoretically better than OLS but not compared to MM. This probably happens because LTS does not use imputed data, but just trims contaminated cells, while the MM tries to fill the cells characterized by weird values with more coherent values. Then, through the analysis of the p-value we can spot different behaviors. The classical model attributes significance to “pctasian”, “pctbachdeg25_over” and “pctpubliccoveragealone” (NOTE: before using vif when considering all variables we got also “medianage”, this was a consequence of the distortion driven by the pull of extreme outliers). On the other hand, the robust estimate finds other drivers of cancer mortality like “pctwhite”, “pctunemployed16_over” and “birthrate”. This shows that the the robust estimate was able to correct the classic OLS. We can also see that the main drivers of cancer mortality are connected to: -poverty and unemployment: in the United State access to preventive screening is reliant on employment based insurances. Due to lack of access a lot of people receive a cancer’s diagnosis when it’s too late and it has reached a terminal stage. -family and social support: researches indicates that living in a married household can influence positively survival rates. This happens because of two possible incomes, psychological support of the other person and sometimes because of a shared health insurance. -systematic disparities: usually counties with an higher proportion of ethnic minorities (a lower “pctwhite”) suffer from healthcare disparities. They have less efficient hospitals, linguistic barriers and delayed diagnosis.